



UNIVERSIDAD
NACIONAL
DE COLOMBIA



Sesión 5: Pipelines de Datos y Lazy Evaluation

Semillero en Ciencias de la Computación

Juan Pedrozo Joel Ostos

Mayo 29, 2026

1. El Modelo Mental de los Pipelines
2. La pieza faltante: mapcat
3. Evaluación Perezosa (Lazy Sequences)
4. Ejercicios Prácticos



1. El Modelo Mental de los Pipelines

El Problema de Componer Funciones

- En sesiones anteriores vimos **map**, **filter** y **reduce**.
- Son herramientas poderosas para transformar datos inmutables.
- Pero, ¿qué pasa cuando necesitamos aplicar múltiples transformaciones en secuencia?
- Recordemos la composición de funciones $f(g(x))$ que vimos la sesión pasada. En programación funcional, esto se traduce en anidación.



Imaginemos que queremos:

1. Tomar una lista de números.
2. Elevarlos al cuadrado.
3. Filtrar solo los que sean impares.
4. Sumar todos los resultados

El código anidado se vería así:

```
(reduce +  
  (filter odd?  
    (map #(* % %) [1 2 3 4 5 6]))))
```

¿Cuál es el problema aquí? ¡El flujo de lectura choca con el flujo de evaluación!



- Para leer el código anterior, nuestros ojos tienen que buscar la estructura más profunda (`map...`).
- Luego saltar a la izquierda hacia (`filter...`).
- Y finalmente saltar nuevamente a la izquierda hacia (`reduce...`).
- Esto no es escalable ni mantenible en procesos complejos de transformación de datos.



Usualmente, en programación funcional sugiere modelar los programas secuenciales como tuberías de fábricas.

- Los datos entran por un extremo (materia prima).
- Pasan por estaciones independientes.
- Cada estación (map, filter) hace un trabajo específico y pasa la nueva colección a la siguiente.
- Se prioriza el **QUÉ** sobre el CÓMO.



Clojure nos ofrece un macro llamado `thread-last` o "flecha doble"(->>).

- Toma una expresión inicial.
- La inserta dinámicamente como el **último argumento** de la siguiente función.
- Pasa el resultado de eso como el último argumento de la que sigue, y así sucesivamente.



- En Clojure, por diseño, las funciones que operan sobre secuencias (`map`, `filter`, `reduce`, etc.) toman la colección como su **último argumento**.
- Esto no es casualidad, está pensado para componer de manera hermosa con `->`.

```
(map f coll)
(filter pred coll)
(reduce f init coll)
```

Nuestro código anterior:

```
(reduce + (filter odd? (map #(* % %) [1 2 3 4 5 6])))
```

Se convierte en:

```
(->> [1 2 3 4 5 6]  
      (map #(* % %))  
      (filter odd?)  
      (reduce +))
```

¡Ahora se lee orgánicamente de arriba hacia abajo!

2. La pieza faltante: mapcat



Sabemos que `map` transforma una colección de elementos aplicando una función uno a uno.

$$f : x \rightarrow y$$

Por lo tanto, al mapear sobre $[x_1, x_2, x_3]$ obtenemos $[y_1, y_2, y_3]$.

Pero, ¿qué sucede cuando la función f **devuelve otra colección**?

El Problema de las Colecciones Anidadas



Imagina que tenemos un registro de estudiantes del semillero y los lenguajes que manejan.

```
(def estudiantes  
  [{:nombre "Juan" :lenguajes ["Haskell" "Clojure"]}  
   {:nombre "Tomas" :lenguajes ["C" "C++"]}])
```

Queremos una lista plana con **todos** los lenguajes.



Si usamos map:

```
(map :lenguajes estudiantes)  
;; (["Haskell" "Clojure"] ["C" "C++"])
```

Obtenemos una secuencia de vectores (lista de listas). Si queremos procesar los lenguajes, esta estructura anidada rompe nuestro pipeline.



- En otros lenguajes funcionales, esta operación se conoce como `flatMap`.
- En Clojure se llama `mapcat`.
- Lo que hace es aplicar una función a cada elemento (como `map`) y luego aplastar o aplanar el resultado un nivel (como `concat`).

¿Cómo funciona por debajo?



Podemos entender `mapcat` como la composición directa de `apply`, `concat` y `map`.

```
(defn mi-mapcat [f coll]
  (apply concat (map f coll)))
```

Nota: La implementación real en Clojure es ligeramente más compleja para mantener la pereza (lazy evaluation), pero semánticamente es idéntica.



Ahora, usando `mapcat` con nuestros estudiantes:

```
(mapcat :lenguajes estudiantes)  
;; ("Haskell" "Clojure" "C" "C++")
```

Hemos aplanado la estructura, eliminando la barrera entre las colecciones internas, dejándola lista para el siguiente paso del pipeline.



Ejemplo integrador: Obtener lenguajes **únicos** en mayúsculas.

```
(->> estudiantes
      (mapcat :lenguajes)
      (map clojure.string/upper-case)
      (set))
;; => #{"HASKELL" "C" "C++" "CLOJURE"}
```

3. Evaluación Perezosa (Lazy Sequences)

Pensemos en la eficiencia de nuestros pipelines.

- Si tenemos una lista de un millón de usuarios.
- Hacemos un `map` para modificar sus correos.
- Luego un `filter` para quedarnos con cuentas activas.

En la evaluación tradicional estricta (**eager**), se crea un nuevo vector de 1 millón de elementos, y luego otro vector resultante.
¡Pico masivo de RAM!

Eager (Estricta): Calcula todo inmediatamente. En el momento en que se llama a la función, el procesador quema ciclos para resolver cada elemento. **Lazy (Perezosa):** Retrasa el cálculo hasta el último segundo posible. Promete que te dará los valores, pero no los procesa hasta que explícitamente se lo exiges (por ejemplo, para imprimirlos en pantalla o guardarlos en base de datos).



En ciencias de la computación teórica, esto se logra mediante "Thunks". Un thunk es utilizado para inyectar un cálculo adicional en otro lugar. En Clojure, una Lazy Sequence es una estructura de datos donde el elemento de resto (la cola) no es un valor, sino una **función sin argumentos** que sabe cómo generar el siguiente elemento.



En Clojure, `map`, `filter`, `mapcat` (y muchas más) producen secuencias perezosas por defecto.

- Cuando haces un pipeline largo, Clojure **no** recorre la colección n veces.
- En su lugar, pide el elemento 1. Lo pasa por el `map`, luego por el `filter`. Si sobrevive, lo entrega.
- Esto unifica conceptualmente la transformación de un elemento único y la iteración sin malgastar memoria.



Si no calculamos todo de inmediato, las estructuras de datos pueden tener un tamaño infinito. ¡Podemos modelar matemáticas continuas!

```
(range)
;; Representa todos los numeros enteros desde 0 hasta
   infinito.
```

¡Ojo! Si intentas imprimir (range) completo en el REPL, el entorno intentará evaluarlo y consumirá toda la memoria.



Para trabajar con secuencias infinitas usamos funciones limitadoras, principalmente take y drop.

```
;; Queremos los primeros 5 enteros pares del infinito  
(->> (range)  
      (filter even?)  
      (take 5))  
;; => (0 2 4 6 8)
```

Otras secuencias infinitas: repeat y cycle



```
(take 3 (repeat "Hola"))  
;; => ("Hola" "Hola" "Hola")
```

```
(take 7 (cycle [1 2 3]))  
;; => (1 2 3 1 2 3 1)
```

cycle es extremadamente útil para rotar turnos, y/o colores de gráficos.

Generación basada en funciones:

iterate



`iterate` toma una función y un valor inicial, y devuelve una secuencia infinita: $x, f(x), f(f(x)), \dots$ Potencias de 2:

```
(take 5 (iterate #(* % 2) 1))  
;; => (1 2 4 8 16)
```

Implementando nuestro propio Infinito



Podemos usar el macro `lazy-seq` para definir recursión que suspende su evaluación. **La sucesión de Fibonacci:**

```
(defn fib []  
  (map first (iterate (fn [[a b]] [b (+ a b)]) [0 1])))
```

- `iterate` genera una secuencia infinita y perezosa aplicando repetidamente la función de transición a partir del estado inicial `[0 1]`.
- La función anónima `(fn [[a b]] [b (+ a b)])` calcula el siguiente par acumulador de manera diferida.
- `map first` transforma la secuencia de pares extrayendo el primer elemento (a) **únicamente cuando es requerido** (por ejemplo, al evaluar un `take`).

```
(defn fib []  
  (map first (iterate (fn [[a b]] [b (+ a b)]) [0 1])))  
  
(take 7 (fib))  
;; => (0 1 1 2 3 5 8)
```



- **Tiempo:** Procesar un elemento a través del pipeline es $O(n)$, pero podemos abortar temprano si usamos `take`, convirtiendo búsquedas pesadas en $O(k)$ donde k es el elemento encontrado.
- **Espacio:** Evaluando $O(1)$ a la vez. No hay duplicación de listas (Garbage Collector limpia elementos ya procesados a medida que avanzamos).

¿Dónde brilla esto en la industria?

- **Data Analytics:** Procesar archivos de logs de 50GB en máquinas con 8GB de RAM. Se procesan línea por línea.
- **Data Scraping:** Paginación infinita. Tratar las páginas web o llamadas a una API rest como un flujo continuo hasta que un predicado sea falso.

4. Ejercicios Prácticos

Dada la siguiente colección de mapas:

```
(def txs [{:monto 1000 :tipo :deposito}
           {:monto -50  :tipo :retiro}
           {:monto -200 :tipo :retiro}
           {:monto 500  :tipo :deposito}])
```

Objetivo: Usar el macro `->` para obtener la **suma absoluta** de todos los `:retiro` mayores a 100.



Tenemos una estructura jerárquica de archivos:

```
(def sistema
  [{:dir "src" :archivos ["main.clj" "core.clj"]}
   {:dir "test" :archivos ["main_test.clj"]}
   {:dir "assets" :archivos ["logo.png"]}])
```

Objetivo: Usar un pipeline que incluya `mapcat` y `filter` para retornar únicamente los archivos que terminen en `".clj"`.

Reto 3: Jugando con el infinito

Objetivo: Crear una secuencia perezosa e infinita de números impares. Luego, extraer solo los elementos desde la posición 100 hasta la 105.

Pista: Investiga qué hace la función `drop` en combinación con `take`.

¡Gracias!